The Three Models, Explained

All three are trained in `src/train.py` inside a scikit-learn `Pipeline` (`StandardScaler` + `OneHotEncoder` -> classifier), tuned with `GridSearchCV(cv=5)`, and scored on F1 to pick the winner.

Your actual results:

Model	Best hyperparameters	Accuracy	Precision	Recall	F1
Logistic Regression	`C=0.01`	0.763	0.470	0.561	0.511
Decision Tree	`max_depth=4, min_samples_leaf=50`	0.755	0.457	0.586	0.514
Random Forest (winner)	`max_depth=10, n_estimators=400`	**0.775**	**0.492**	**0.608**	**0.544**

---

1. Logistic Regression — the baseline

How it works, in plain terms: It learns one weight per feature and adds them up (`w1*x1 + w2*x2 + ... + b`), then squashes the result into a 0–1 probability with the sigmoid function. It's fundamentally drawing a single straight decision boundary (in however many dimensions your features live in) between "default" and "no default."

Why include it: It's fast to train, the coefficients are directly interpretable (a positive weight on `NUM_DELAYED_PAYMENTS` literally means "more delayed payments -> higher predicted default probability"), and it's the standard baseline every other model needs to beat to justify its extra complexity.

The one hyperparameter tuned: `C`, the inverse of regularization strength. A small `C` (the winner here, 0.01) means *stronger* regularization — the model is discouraged from assigning very large weights to any one feature, which helps it generalize instead of overfitting to noise in the training set.

Why it came in last on F1 (0.511): Real default behavior isn't a straight line. Say the EDA plot in `01_eda_plots_explained.md`: default rate vs. delayed payments climbs steeply after 3-4 months but is relatively flat 0 vs. 1. A linear model struggles to capture that kind of curve/threshold behavior as well as a tree-based model can.

---

2. Decision Tree — a single tree of yes/no questions

How it works, in plain terms: It repeatedly splits the data by asking yes/no questions on one feature at a time — e.g. "is `NUM_DELAYED_PAYMENTS`

> *2?" — choosing the split at each step that best separates defaulters from non-defaulters. The result is a flowchart: follow the questions down to a leaf, and that leaf's majority class (or probability) is the prediction.*

Why include it: It can capture non-linear patterns and interactions a linear model can't (e.g. "high utilization AND several delayed payments" can matter more than either alone), and — unlike Random Forest — a single small tree can actually be drawn and read by a human. That interpretability is valuable in lending, where regulations often require an explainable decision.

The hyperparameters tuned: `max_depth` (how many questions deep the tree can go) and `min_samples_leaf` (how many training examples a leaf must have to be considered). Both are overfitting controls. The winning tree is shallow and conservative — `max_depth=4`, `min_samples_leaf=50` — because GridSearchCV found that deeper/more granular trees memorized noise in the training data and did worse on the held-out test set.

Why it beat Logistic Regression on F1 (0.514 vs 0.511) but barely: A single tree captures non-linearity, but it's also high-variance — small changes in the training data can produce a very different tree. It's a stepping stone to Random Forest, not the end goal.

---

3. Random Forest — the winner

How it works, in plain terms: Train *many* decision trees (400 of them here), each one on a random bootstrap sample of the training rows and a random subset of features at each split, then average their predicted probabilities. Each individual tree is often weak and overfit on its own random slice of data, but averaging many of them cancels out most of that noise — "the wisdom of the crowd" applied to decision trees.

Why it won: It gets the same ability to model non-linear patterns that a single Decision Tree has, but the averaging across 400 trees makes it far less prone to overfitting any one quirk of the training data. That's exactly what the numbers show — better accuracy (77.5%), better precision (0.49), better recall (0.61), and better F1 (0.544) than either of the simpler models, on data the model never trained on.

The hyperparameters tuned: `n_estimators` (how many trees — more generally helps up to a point, at the cost of training time) and `max_depth` (same overfitting control as the single tree, but less critical here since averaging many trees already fights overfitting).

The honest trade-off: Random Forest is harder to explain to a non- technical stakeholder than a single Decision Tree or Logistic Regression's coefficients — "it averages 400 trees" is a less satisfying answer than "if utilization is over 80%, flag it." In a regulated lending context, that's a real trade-off between accuracy and explainability, and it's worth mentioning that teams sometimes pick a slightly weaker but more explainable model for exactly that reason (or add a tool like SHAP on top, which this project intentionally left out to keep things simple — see `04_interview_qa.md`).

Why compare three models instead of just picking one

Because you don't know in advance which inductive bias fits a given dataset best. Linear boundary? Non-linear but low-variance single tree? Non-linear and lower-variance ensemble? The only way to know is to try a small spectrum of complexity and measure — which is exactly what the F1 column above does.